

Alien 内核的 x86_64 移植与 vDSO 引入

摘 要

传统内核因为各个模块高度绑定，一处故障很容易扩散到整个系统，导致可靠性降低和维护难度增加。Alien操作系统通过将内核功能拆分为相互隔离的“故障域”，并使用Rust语言的安全机制，在操作系统层面提供更高的容错并缩小漏洞影响范围。x86_64架构在服务器和桌面领域使用广泛、生态成熟，且用户态中断机制已在真实硬件上落地。移植能更好地验证隔离设计的可用性，同时引入新的软件和硬件特性。x86_64在引导、中断控制、Linux ABI与设备发现等机制上与RISC-V存在大量差异，移植到x86_64需要修改整体框架以支持多架构扩展，并添加大量x86_64细节。

为此，本文先分析了Alien原始设计中的域模型，随后设计并实现了x86_64的启动流程、trap/syscall入口、内存管理及设备探测与驱动加载框架。在此过程中，本文提炼出了Trap分发机制等公共抽象，让架构差异仅存在于架构实现内部，使得主逻辑可以跨平台。在此基础上，本文进一步设计并集成了vDSO机制：在保证系统调用路径仍然保留的前提下，通过将一块只读的代码与数据区域映射至用户地址空间，使用户程序获取当前时间可以绕过系统调用。

本文完成了AsyncAlien在x86_64上的包含启动、多核同步、设备发现、域加载及进入用户态的全流程，验证了功能正确性。性能测试结果显示，引入vDSO后，clock_gettime等时间查询操作的开销降低约80%。本工作不仅完成了Alien架构移植与功能扩展，也为构建多架构的内核提供了设计参考与实现经验。

关键词：操作系统；多架构支持；内存安全；vDSO；系统调用

Porting the Alien Kernel to x86_64 and Introducing vDSO

Abstract

Traditional kernels suffer from high module coupling, where a single failure can easily propagate throughout the system, reducing reliability and increasing maintenance difficulty. The Alien operating system enhances fault tolerance at the OS level by partitioning kernel functions into isolated "fault domains" and leveraging Rust's safety mechanisms to limit the scope of vulnerabilities. The x86_64 architecture is widely used in servers and desktops with a mature ecosystem, and its user-space interrupt mechanism has been implemented on real hardware. Porting allows for better validation of isolation design feasibility while introducing new software and hardware features. However, x86_64 differs significantly from RISC-V in mechanisms like booting, interrupt control, Linux ABI, and device discovery. Porting to x86_64 requires modifying the overall framework to support multi-architecture extensions and incorporating extensive x86_64-specific details.

To this end, this paper first analyzes the domain model in the original design of Alien, then designs and implements the x86_64 boot process, trap/syscall entry, memory management, and device detection and driver loading framework. During this process, it extracts common abstractions such as the Trap dispatch mechanism, ensuring architectural differences exist only within the implementation of the architecture, allowing the main logic to be cross-platform. Building on this, the paper further designs and integrates the vDSO mechanism: while maintaining the system call path, it maps a read-only code and data region to the user address space, enabling user programs to bypass system calls when retrieving the current time.

This paper completes the full process of AsyncAlien on x86_64, including booting, multi-core synchronization, device discovery, domain loading, and entry into user mode, verifying the correctness of the functions. Performance test results show that with the introduction of vDSO, the overhead of time-related operations such as `clock_gettime` is

reduced by approximately 80%. This work not only achieves the porting and functional expansion of the Alien architecture but also provides design references and implementation experience for building a multi-architecture kernel.

Key Words: Operating System; Multi-architecture Support; Memory Safety; vDSO; system call

目 录

摘 要	I
Abstract	II
第 1 章 引言	1
1.1 研究背景与意义	1
1.2 研究目标与问题陈述	1
1.3 本文的组织结构	2
第 2 章 背景与相关工作	3
2.1 原始 Alien 系统综述	3
2.2 微内核与隔离设计对比	3
2.3 Rust 语言的内存安全与共享堆实践	4
2.4 vDSO 与高频路径优化	4
第 3 章 系统架构与设计	6
3.1 引入 x86_64 并屏蔽多架构差异	6
3.1.1 引导与早期初始化	6
3.1.2 内存管理	6
3.1.3 中断控制	7
3.1.4 设备探测与域加载	7
3.2 vDSO 设计	8
3.2.1 vdso_crate_template 设计	8
3.2.2 vDSO 与 syscall 设计	9
第 4 章 x86_64 移植与实现细节	10
4.1 引导与早期初始化	10
4.2 x86_64 与 RISC-V 的页表结构与寄存器差异	12
4.3 trap/syscall 初始化	14
4.5 ACPI/AML 与 PCI 的探测方式	16
第 5 章 vDSO 实现与用户态集成	18
5.1 vdso_crate_template 构建链路与产物	18
5.2 内核侧 MemIf 实现与 vDSO 装载	19

5.3 内核侧映射与刷新策略	20
5.4 userlib vdso 加载	21
第 6 章 验证与评估	23
6.1 测试设计	23
6.2 功能验证与运行证据	24
6.3 性能度量与分析	25
7. 下一步研究方向	26
7.1 协作调度完善（引入 vsched2）	26
7.2 引入用户态中断	26
7.3 在 x86_64 物理机上运行	26
结 论	28
参考文献	29

第1章 引言

1.1 研究背景与意义

随着计算机系统功能日益复杂，Linux内核代码规模也在不断增长。传统宏内核架构将操作系统的大部分功能集中在单一的执行环境中，任何一个模块出问题，都可能影响整个系统的安全性和可靠性。从多个真实漏洞来看，传统内核的复杂性不只是代码量大、逻辑耦合紧，更导致漏洞修复代价高、验证困难、攻击面宽且难以收敛。

为应对上述风险，Linux内核社区提出了严格的内核内存权限控制、KASLR、seccomp、模块签名等防御手段。这些机制有效提高了攻击门槛，但其本质仍是对复杂系统的被动补救，难以从根本上降低内核复杂性带来的风险。隔离式内核架构由此成为一种可行的解决方案：通过把系统功能划分为相对独立的故障隔离域，各域通过明确接口交互，避免直接共享内核内存，从而将故障影响局限在域内并减少受影响的内核路径。

在这种架构下，系统具备如下优势：

1. 故障局限化：单个域的漏洞或故障不会导致整个系统崩溃
2. 攻击面缩减：每个域只暴露必要的接口，减少高风险路径
3. 独立演进：功能模块可以独立升级或替换，降低系统整体维护成本
4. 隔离边界明确：安全属性更易于验证与审计

Alien是基于域隔离理念的内核实现，已在RISC-V平台验证了隔离域、共享堆通信与域加载等机制。x86_64平台在启动、trap处理、内存和设备管理等方面与RISC-V存在显著差异，需要设计平台适配层。系统调用、时间查询等高频路径对性能尤为敏感，传统Linux通过vDSO在用户态提供快速路径以降低开销。任务调度框架作为操作系统的核心组件之一，对于任务整体性能有着至关重要的作用。为了引入基于vDSO的调度框架vsched2，需要先引入vDSO以及快速syscall路径作为前置工作。

1.2 研究目标与问题陈述

本文的研究目标是：在不动现有RISC-V路径的前提下，打通x86_64的运行链路，使系统能够完成启动、早期初始化、设备发现、域加载、多核启动并最终进入用户态；

同时为vDSO、协作调度等高频机制预留好扩展接口，让这个最小可运行版本后续有优化余地。

围绕这个目标，本文需要回答以下2个问题：

- 如何在启动流程、内存布局、中断处理、设备探测机制等方面建立统一的抽象，使x86_64与RISC-V之间实现尽可能多的代码复用，同时保持语义和功能 correctness？
- 如何将vDSO引入Alien，同时不影响现有的域隔离、系统调用等机制，实现一个功能正确的时间获取调用？

本文的研究范围就限定在上面这些问题在x86_64平台上的实现和验证。最后会从功能正确性和性能开销这2个维度，说明迁移后的系统不仅能启动，而且具备一定的实际可用性。

1.3 本文的组织结构

本文后续章节按背景、实现、验证的顺序展开。

第二章介绍Alien的原始设计以及相关研究进展，包含域隔离、共享堆通信、vDSO和协作调度等核心概念，为后面的设计提供理论基础。

第三章给出系统整体架构与设计要点，说明平台抽象、虚拟地址空间和高频路径的统一设计思路，以及如何在设计中为vDSO和协作调度预留扩展接口。

第四章重点讲x86_64的移植和实现细节，依次说明引导与早期初始化、物理与虚拟内存管理、多核启动同步、trap与syscall的适配、任务切换、设备发现以及驱动域加载的工程实现和关键取舍。

第五章描述vDSO的构建、vVAR布局与无锁一致性策略、用户态映射与auxv注入，以及快路径和回退机制的协同实现。

第六章给出实验验证与评估方法，包括功能正确性测试、隔离性与恢复性验证，以及针对syscall和vDSO的性能基准测试与分析。

第七章讨论下一步研究方向和改进方向。

第2章 背景与相关工作

2.1 原始 Alien 系统综述

Alien的核心思路不是简单的模块化，而是把内核能力按职责和功能拆成多个故障隔离域，再让它们通过显式接口协作。传统宏内核是“各模块共享同一个高权限执行空间”，Alien的设计则直接在框架上把访问边界、调用边界和故障边界划分清楚，这样系统在局部出问题，其它部分不容易受影响，大概率仅需要修复局部即可。本文的迁移工作不能破坏这个设计，并且应优先复用那些已经验证过的域运行时机制。

从运行角度来看，Alien 的内核功能以一组可动态管理的隔离域形式存在，这些域作为内核模块支持运行时加载、卸载与热更新。每个域通过显式的域接口对外提供服务，域内部依赖安全抽象实现其功能；域管理运行时负责域的装载、回收与替换等治理动作，从而在运行时形成“加载—通信—治理”的闭环。该闭环把隔离、故障隔离与演进能力作为设计核心：通过语言级的类型与内存安全约束和清晰的接口边界，既允许域独立演进与热替换，又避免跨域协作退化为难以观测和验证的隐式耦合。

在域间通信上，Alien采用控制面和数据面分离的方式。控制路径使用受限接口和轻量调用，数据路径借助共享堆来传递对象。这种设计兼顾了性能和边界可验证性：一方面降低了高频数据交换的复制成本，另一方面通过受控类型和生命周期约束，减少了直接共享内存带来的越界访问和对象悬挂风险。Alien不依赖人工约定来维持正确性，而是尽量把约束落到接口和类型边界上。

不过，原始Alien的实现主要面向RISC-V平台，它的启动链路、异常入口、设备发现和中断模型都带有明显的平台特征。这在原型阶段有利于快速验证域机制的可行性，但同时也导致部分实现与平台细节耦合较深。因此，本文所做的x86_64迁移并不是另起炉灶，而是在保留域机制主干的前提下，把平台相关差异从上层运行时中收敛出去，逐步建立起一套可复用的跨架构实现骨架。

2.2 微内核与隔离设计对比

微内核体系[13]（如seL4[12]）通过最小化内核功能来压缩可信计算基，并依赖消息传递实现服务协作，隔离边界天然更强。与此相对，Alien采取的路线更偏向工程折中：不追求一次性重写全部系统服务到用户态，而是在现有内核形态上逐步引入

隔离域和显式接口，采用"渐进式隔离重构"的策略。两者的共同目标是降低故障扩散和攻击面，不同点在于实施路径与成本结构。

从设计粒度看，微内核追求最小内核，通常只保留地址空间、线程和进程间通信等最基础的服务，其他功能以服务进程形式运行在用户态。这种设计天然具有强隔离但引入的上下文切换和消息传递开销也较高。**Alien**则保留了大部分内核功能在内核态执行，但通过域机制加强了模块间的边界与可验证性，兼顾了隔离效果与性能开销。

对本研究来说，微内核提供了隔离设计的参照系与终极目标，而**Alien**路线强调迁移可行性与渐进演化能力，更贴合当前代码基础与实践条件。这一权衡决定了后续x86_64迁移的实施策略：优先复用现有运行时框架，在其上引入平台抽象和隔离边界，而非彻底推翻重来。

2.3 Rust 语言的内存安全与共享堆实践

Rust的所有权、借用和生命周期机制[15]为隔离域系统提供了更强的静态约束基础。传统C实现的隔离系统往往依赖开发者手工维护引用计数、对象归属关系和访问权限，一旦出错就难以追踪。而在**Alien**中，共享堆通信并非"自由共享内存"，而是通过受控数据结构和接口边界传递对象，尽量把对象归属、可变性和回收责任写进类型语义。这一实践可显著降低双重释放、悬空引用和越界访问等常见错误的发生概率。

从编译器视角看，**Rust**的类型系统和生命周期检查能够在编译期捕获大量内存安全问题，避免运行时崩溃或不确定行为。这对隔离域系统尤其关键：域间对象传递不能依赖运行时检查，必须在编译期形成强保证。例如，当某个对象从域A转移到域B时，**Rust**的所有权转移规则保证了A不会再访问该对象，从而避免了域间的竞态条件和对象悬挂。

同时，**Rust**并不意味着底层代码可以完全避免不安全操作。域加载、地址空间映射、硬件交互等路径仍需要少量**unsafe**实现，这些代码触及底层内存和硬件语义，超出了**Rust**安全检查范围。相关工作的意义在于说明：**Rust**能够把大多数错误压缩到少数可审计边界，从而使隔离域系统的工程质量更可控，而不是宣称"语言本身自动解决所有问题"。这样的分层使得代码审计、测试和验证都更加聚焦有效。

2.4 vDSO 与高频路径优化

vDSO（virtual dynamic shared object）是Linux在用户态提供高频服务快速路径的经典机制，其核心目标是在保持语义正确的前提下降低陷入内核的开销。典型应用包括时间查询（`clock_gettime`）、时间戳获取等操作，这些服务频率高但通常无需内核状态修改。通过把只读数据映射到用户态并提供快速计算路径，vDSO可以显著减少系统调用开销。

对Alien而言，vDSO不能被视为独立优化补丁，而应与域隔离、系统调用回退和调度语义协同设计。若快速路径与主链路割裂，可能带来语义偏差甚至边界绕过风险。例如，若vDSO中的时间查询与调度器维护的系统时间不一致，可能导致应用层观察到时间倒流；若vDSO快速路径绕过了域隔离边界，则也就破坏了隔离承诺。因此，vDSO的正确性不仅要求快速路径算法本身正确，还要求与整个系统的隔离、调度和一致性框架协同。

本研究借鉴vDSO思想的重点并非“照搬实现”，而是建立“快慢路径协同”的设计原则。这一原则意味着：高频、低风险、状态不变的服务优先走用户态快速路径以降低延迟；当快速路径条件不满足（如需要修改状态或缺乏必要信息）或一致性检查失败时，必须无缝回退到系统调用慢路径并由内核完成。这样的设计既保证了性能收益，也保证了语义正确性和域隔离不被绕过。该原则为后续x86_64实现章节提供了性能与正确性并重的设计基线。

第3章 系统架构与设计

本文在这一章给出总体架构和关键设计要点，目标是在尽可能屏蔽架构差异的情况下引入x86_64平台相关实现，完成x86_64运行链路，然后在此基础上实现vDSO的引入和测试。

3.1 引入 x86_64 并屏蔽多架构差异

设计目标是把架构相关的差异限制在可控的几处，从而使上层业务逻辑在统一的抽象之上工作。为此把移植工作拆分为四部分：引导与早期初始化、内存管理、中断控制、设备探测与域绑定。每一部分收敛平台差异，向上层提供统一接口，内部按架构实现。

3.1.1 引导与早期初始化

引导层的策略是让不同平台通过不同的启动汇编进入统一的Rust编写的main函数，并传递引导信息指针。具体做法是约定一个统一的boot_info参数，传递物理内存布局、initrd基址、可用内存信息等数据，留给后续部分使用。这个信息以指针的形式存在，在不同平台上可能有不同的实际含义。在x86_64上，Multiboot引导完成后处于32位保护模式，需要设置页表，设置CR0、CR4、EFER控制寄存器进入长模式，然后再进入main；而RISC-V侧只需要在SBI初始化完成后压入boot信息指针后进入main即可。随后双架构初始化percpu数据寄存器（x86_64上是gs，RISC-V上是gp）并进入早期初始化。x86_64侧在早期初始化中设置APIC打开本地中断、启用浮点/SSE功能，建立早期时间模块。而RISC-V得益于其简洁的设计，不需要进行相关工作。早期初始化的设计将更多平台差异隐藏到具体实现中。

3.1.2 内存管理

在物理内存管理上，x86_64使用Multiboot信息获取可用物理内存区域，RISC-V解析FDT的/memory节点获取可用物理内存信息，在去除低地址区域和内核ELF占据的物理页后，物理页帧管理器将剩余可用物理内存按页划分，构建空闲页帧池，并提供统一的分配与回收接口。物理页帧回收工作可以借助Rust语言提供的RAII特性进行自动回收。内核链接脚本中预留了内核堆空间，这部分物理内存交给内核堆分配器进行分配。由于内核堆空间在内核ELF区域内，故不会与物理页帧分配器冲突。

在虚拟内存管理上，核心是定义一个跨架构的VSpace抽象，包含如下操作：获取页表根地址、创建/销毁地址空间、映射与取消物理页映射、查询虚拟地址到物理地址的映射。每种架构提供对应的页表项和页表寄存器实现：在riscv上，适配器实现Sv39语义并通过satp切换页表根；在x86_64上，适配器实现四级页表语义并通过CR3切换页表根。上层使用了VSpace抽象操作而不是直接操作寄存器或页表条目，从而屏蔽页表层级和寄存器的差异。域加载、vDSO映射以及用户地址空间的初始化都通过该抽象完成。

3.1.3 中断控制

中断控制采用两层设计：汇编入口负责保存和恢复中断上下文、切换栈和地址空间，然后把控制权交给用Rust实现的中断分发函数。对于RISC-V，故障与系统调用共享中断路径，仅需要区分内核中断与用户中断路径。x86_64引入了syscall指令和syscall MSR，需要额外区分故障和系统调用路径。故障处理时，部分寄存器的保存和恢复由CPU自动完成，剩余逻辑与RISC-V类似。系统调用则完全由软件保存和恢复所有寄存器，本文的做法是将percpu区域映射到用户态高地址，先暂存用户栈指针到内存中再恢复，以此完成换栈操作。中断分发函数负责解码异常原因、调用相应的处理逻辑并将结果写入用于恢复的中断上下文。这部分受架构影响较大，分别实现更好。

3.1.4 设备探测与域加载

设备探测职责在于把平台发现设备的细节隐藏到平台实现中，对外只暴露出探测得到的设备信息。实现上，RISC-V使用FDT进行设备解析，x86_64则使用ACPI/PCI机制。不同平台存在不同平台的专有设备，例如PLIC、Goldfish等，即使是相同设备也存在不同的I/O方式，例如MMIO和PIO。在本文的设计中，对于平台专有设备的驱动域，假定在指定架构下实现，其它架构尝试编译会报错；对于平台共用设备的驱动域，其实现中支持多种I/O方式，域加载时需要按架构初始化I/O方式，这样就能隐藏架构差异，复用同一段业务逻辑代码。上述两类域需要判断当前架构后加载。对于内核组件域中架构相关的部分，在实现中统一域接口操作，内部按架构实现。除此之外，还存在大量架构无关的内核组件域和虚拟设备域，这些域可以直接加载。由此便完成了设备探测与域加载的平台无关上层抽象。

总体上，引入x86_64的关键在于把架构专有步骤封装在少数几个边界：引导和早期初始化、VSpace内部实现、汇编中断入口和中断分发函数、设备探测实现与域加载架构专有路径。只要这些边界的约定清晰且稳定，上层业务逻辑便无需了解底层的寄存器、页表或异常细节，从而实现跨架构的代码复用与可维护性。

3.2 vDSO 设计

本小节分为两部分：首先说明vdso_crate_template的设计目标与生成器职责，然后论述vDSO与syscall的设计与回退语义。

3.2.1 vdso_crate_template 设计

vdso_crate_template[5]的核心职责是生成符合vDSO[11]要求的动态链接库，并提供内核和用户库的加载和访问接口。它不直接承担具体的物理内存和用户页表管理，而是把这些职责抽象为接口，由内核去完成。

在vdso_crate_template的设计中，数据分为共享数据和私有数据。从地址空间的角度看，共享数据可以被所有地址空间访问，而每个地址空间只能访问自己的私有数据。

生成器在构建时输出两类主要结果，即vDSO共享对象和静态API库。内核可以通过API库自动处理vDSO代码和数据的加载和重定位。用户库，可以通过API库像普通函数调用一样调用vDSO函数，因为API库记录了函数API及其在vDSO的偏移，只要用户提供了auxv提供的vDSO起始地址，就可以计算出vDSO函数地址。

在实际映射流程中，生成器通过一个内存接口由内核完成地址与物理页的分配与映射。这个接口把vDSO加载过程抽象出分配虚拟地址、分配物理页、页映射、权限更改和虚拟地址翻译这几个内核实现，使生成器逻辑能够同时适配内核初始化与映射到用户地址空间两种场景。

MemIf接口与功能对应如下：

表 3-1 MemIf 接口设计

接口名	功能说明
MemIf::valloc	在指定地址空间中预留一段虚拟地址区域供 vVAR 与 vDSO 使用，不立即分配物理页，返回该区域的基址。
MemIf::ppage_alloc	分配连续的或若干物理页用于存放 vVAR 或 vDSO 数据，返回物理页句柄。
MemIf::ppage_clone	复制物理页句柄以便在不同地址空间中安全地复用相同物理页，或按实现进行引用计数处理。

<code>MemIf::map</code>	将指定的物理页句柄映射到目标地址空间的虚地址，并设置只读、可执行、用户可见等权限标志。
<code>MemIf::change_protect</code>	在映射完成后修改该虚地址区间的权限，用于把 vVAR 先以可写方式初始化后切换为只读，或把代码段设为只读可执行。
<code>MemIf::get_kernel_vaddr</code>	在内核上下文中获取映射到用户地址的内核可访问虚拟地址，便于内核在初始化阶段直接向 vVAR 写入数据或验证映射内容。

内核端在首次加载时完成完整的ELF解析、段拷贝和重定位，在后续为其他地址空间建立映射时可以复用已加载的物理页。为了兼容物理页句柄通过Drop trait自动释放的设计，MemIf还提供了ppage_clone用于延续物理页生命周期。

3.2.2 vDSO 与 syscall 设计

本文为测试vDSO功能，在vDSO中设计了clock_gettime和gettimeofday的快速路径实现。这两个函数不存在私有状态，全部状态均存在于共享区。

在clock_gettime的总体设计中，vDSO内部只负责从共享区中读取当前时间并把结果转换为用户态所需的格式，不参与时间维护。

gettimeofday与clock_gettime共享同一套时间快照，实现逻辑类似，都是只从共享区内读取时间值，转换为对应的时间格式。这也是实现这两个系统调用的原因。

内核侧的刷新逻辑负责维护这份共享时间快照。内核在时钟中断读取当前时间，再把最新值写入共享区，并通过原子变量标记告知用户是否正在更新。用户态读取时先检查更新是否完成，再读取时间并在必要时重试，从而避免读到更新中的脏数据。由此，clock_gettime和gettimeofday在正常情况下都可以直接命中vDSO快路径，而当共享区暂不可用、内核正在更新中或其它异常情况时，统一回退到syscall，保证语义完整和运行稳定。

内核在进程初始化时先建立vDSO和共享时间区域的映射，再通过auxv把vDSO入口暴露给用户态解析，用户态进行缓存。随后内核持续刷新共享时间快照，使用户态始终能够通过同一套vDSO接口获得较低开销的时间查询结果。

第 4 章 x86_64 移植与实现细节

本章描述将系统从RISC-V迁移到x86_64时的实现要点，聚焦启动与早期初始化、percpu与早期初始化、页表与寄存器差异、trap/syscall的切换语义、以及ACPI/PCI的探测方法。

4.1 引导与早期初始化

为了使用Multiboot进行引导，首先按照Multiboot1规范[9]，通过链接脚本控制在二进制文件头部嵌入一个Multiboot header，指定ELF加载范围，以及入口点。x86_64的早期启动需启用分页模式，加载早期页表，完成控制寄存器与长模式相关配置，最后ljmp进入64位代码。Multiboot引导完成后，eax中存储着魔数，ebx中是Multiboot信息指针。根据x86_64 ABI，将魔数作为第1个参数移到edi，Multiboot指针作为第2个参数移到esi后，就可以调用Rust函数，校验魔数后进入Rust编写的主函数。

第一步是初始化percpu寄存器和cpuid。引入percpu crate需要在链接脚本中添加一个percpu段。percpu的初始化在预先分配的内存中构造每核数据区并设置percpu寄存器值，以便在陷阱/系统调用时通过GS快速访问。cpuid在RISC-V侧SBI引导后通过寄存器传递给entry，而x86_64侧可以通过封装底层cpuid汇编操作的raw_cpuid crate获取cpuid，然后写入percpu区。自此之后，所有的cpuid获取都通过percpu区获取。

接下来是平台早期初始化。对于x86_64，每个CPU在进入内核运行之前需要初始化FPU/SSE[3]支持并配置本地APIC功能。

表 4-1 FPU/SSE 初始化代码

```
pub fn init_fpu() {
    unsafe {
        // CR0: 清 EM/TS, 置 MP。
        let mut cr0 = control::Cr0::read();
        cr0.remove(control::Cr0Flags::EMULATE_COPROCESSOR);
        cr0.insert(control::Cr0Flags::MONITOR_COPROCESSOR);
        cr0.remove(control::Cr0Flags::TASK_SWITCHED);
        control::Cr0::write(cr0);

        // CR4: 置 OSFXSR/OSXMMEXCPT。
        let mut cr4 = control::Cr4::read();
        cr4.insert(control::Cr4Flags::OSFXSR);
        cr4.insert(control::Cr4Flags::OSXMMEXCPT_ENABLE);
    }
}
```

```

control::Cr4::write(cr4);

// 保留 x87 初始化。
core::arch::asm!("fninit", options(nomem, nostack,
preserves_flags));
}
}

```

首先在CR0上清除FPU模拟位并设置MP，允许协处理器按x86语义工作；随后通过CR4启用OSFXSR与OSXMMEXCPT，从而支持使用FXSAVE/FXRSTOR与SSE的异常模型。执行fninit将x87状态复位为已知值，保证后续浮点指令在已定义的状态下运行。

表 4-2 主核 APIC 初始化代码

```

/// 初始化主核（BSP）的 Local APIC。
pub fn init_primary_apic() {
    // 禁用 8259A PIC
    let _ = x86_apic::port_io::disable_8259a();

    let is_x2apic_mode = cpu_has_x2apic();
    IS_X2APIC.store(is_x2apic_mode, Ordering::Release);

    let xapic_base = if is_x2apic_mode {
        0 // x2APIC 不需要 MMIO 基地址
    } else {
        (unsafe { x2apic::lapic::xapic_base() as usize }) +
        (PHYS_VIRT_OFFSET as usize)
    };

    let mut context = LocalApicContext::new(xapic_base,
is_x2apic_mode)
        .expect("failed to create LocalApicContext");

    // 启用 APIC
    context.enable().expect("failed to enable APIC");

    *APIC_CONTEXT.lock() = Some(context);
    APIC_CONTEXT_READY.store(true, Ordering::Release);
}

```

init_primary_apic在主核上完成 Local APIC 的早期初始化：先通过x86_apic::port_io::disable_8259a()关闭旧式8259A PIC，避免干扰。随后通过

`cpu_has_x2apic()`检测CPU是否支持x2APIC，并把结果写入全局变量`IS_X2APIC`。接下来根据是否启用x2APIC决定`xapic_base`。x2APIC模式不需要MMIO，而是通过MSR访问，因此将`xapic_base`设置为0。否则从`x2apic::lapic::xapic_base()`获取物理地址并加上`PHYS_VIRT_OFFSET`转换为内核虚拟地址。用该基址和模式创建`LocalApicContext`（在`x2apic crate[2]`基础上对APIC的进一步封装），调用`context.enable()`对硬件做寄存器编程，例如设置`timer vector`、配置`timer`等。最后将上下文写入互斥锁保护的`APIC_CONTEXT`并通过原子变量`APIC_CONTEXT_READY`更新就绪状态。

在`x2apic crate`的设计中，`LocalApic`对象仅记录设置项，只有在调用`enable()/enable_timer()`时才会真正写入寄存器。因此，主核创建`LocalApicContext`对象后，从核可以直接在这个对象上调用`enable()`，对自己核上的APIC进行配置。

4.2 x86_64 与 RISC-V 的页表结构与寄存器差异

x86_64使用四级页表（PML4→PDPT→PD→PT），每级512项，索引宽度为9位。RISC-V的Sv39使用三级页表（PGD→PMD→PT），每级512项，索引宽度为9位。

表 4-3 x86_64 与 RISC-V 页表项和页表抽象

```
#[cfg(target_arch = "x86_64")]
pub type ArchPTE = X64PTE;
#[cfg(not(target_arch = "x86_64"))]
pub type ArchPTE = Rv64PTE;

#[cfg(target_arch = "x86_64")]
pub type ArchPageTable<T> = X64PageTable<T>;
#[cfg(not(target_arch = "x86_64"))]
pub type ArchPageTable<T> = Sv39PageTable<T>;

pub struct VmSpace<T: PagingIf<ArchPTE>> {
    table: ArchPageTable<T>,
    areas: BTreeMap<usize, VmAreaType>,
}

pub trait PagingIf<PTE: GenericPTE>: Sized {
    /// 分配 4K 大小的物理页面
    fn alloc_frame() -> Option<Box<dyn NotLeafPage<PTE>>>;
}
```

对于页表项，可以抽象出来的主要操作有：

1. 创建一个指向指定物理页的页表项
2. 设置页表项的权限
3. 获取页表项指向的物理地址和权限

而对于页表，可以抽象出来的主要操作有：

1. 映射虚拟页A到物理页B
2. 修改虚拟页A映射的物理页B
3. 修改虚拟页A映射权限
4. 删除虚拟页A映射
5. 获取页表根目录的物理地址
6. 查询虚拟地址映射的物理地址

在page-table crate的设计下，不同架构可以抽象出相同的页表操作，并进一步通过VmSpace与PagingIf接口实现基于RAII的物理页面分配和自动释放。在VmSpace对象触发drop时，内部包含的通过PageingIf::alloc分配的物理页对象将会自动drop，返还占用的物理地址。

对于不同架构，使用的页表寄存器也各不相同，x86_64使用CR3寄存器，RISC-V使用satp寄存器。切换页表时写入页表寄存器的值的含义也各不相同。

表 4-4 获取写入页表寄存器的值

```
pub fn kernel_page_table_token() -> usize {
    #[cfg(target_arch = "x86_64")]
    {
        kernel_page_table_root_paddr()
    }
    #[cfg(target_arch = "riscv64")]
    {
        8usize << 60 | (kernel_page_table_root_paddr() >>
FRAME_BITS)
    }
}

/// 激活页表（Sv39）。
#[cfg(target_arch = "riscv64")]
pub fn activate_paging_mode(page_table_token: usize) {
    sfence_vma_all();
    satp::write(page_table_token);
    sfence_vma_all();
}
```

```
// 激活页表
#[cfg(target_arch = "x86_64")]
pub fn activate_paging_mode(page_table_token: usize) {
    unsafe {
        Cr3::write(
            PhysFrame::containing_address(
                PhysAddr::new(page_table_token as u64)
            ),
            Cr3Flags::empty()
        );
    }
}
```

如代码所示，在x86_64上，`kernel_page_table_token`返回值直接是内核页表根的物理地址，上层在切换CR3时使用该物理地址即可。在riscv64上，函数将物理页号与标志（这里代表使用Sv39分页模式）合并成一个token，然后写入satp寄存器。对于上层，可以直接使用`activate_paging_mode(kernel_page_table_token())`激活，而不需要关注实现细节。

4.3 trap/syscall 初始化

为了在用户页表隔离场景下仍能在syscall中访问percpu数据，内核在所有虚拟地址空间的高地址映射一份percpu镜像并在初始化时把percpu寄存器设为该镜像的基址。在切换地址空间时，不需要修改GS的值就能正确访问percpu数据。下面给出两处关键实现的真实摘录：percpu镜像的写入与MSR/LSTAR的设置，以及syscall汇编入口中对percpu的访问。

表 4-5 percpu 镜像写入与 LSTAR/SFMask 设置

```
// 在 syscall 初始化时把 GS 指向 percpu 镜像高地址
let gs_value = percpu::read_percpu_reg();
let gs_mirror = PERCPU_MIRROR_BASE - _percpu_start as *const ()
as usize + gs_value;
unsafe { percpu::write_percpu_reg(gs_mirror); }

// 设置 syscall 相关 MSR，将快速入口指向 trampoline 内的
syscall_entry
let lstar_offset = syscall_entry as *const () as usize -
strampoline as *const () as usize;
let lstar = VirtAddr::new((TRAMPOLINE + lstar_offset) as u64);
```

```
LStar::write(lstar);
SfMask::write(sfmask);
unsafe { Efer::write(Efer::read() |
EferFlags::SYSTEM_CALL_EXTENSIONS); }
```

上述片段的目的是在内核虚拟地址空间为percpu数据建立一个稳定且可由汇编访问的高地址镜像，并将该镜像地址写入寄存器镜像或相关MSR，保证swaps后汇编可以通过GS快速定位。与此同时，将LSTAR设置为trampoline的入口地址，使得用户态的syscall指令会直接跳入内核预期的trampoline代码段；SfMask的配置则用于屏蔽sysret/syscall交替过程中不应被继承的标志位（如中断使能），从而减少返回时的时间窗风险。

表 4-6 syscall 汇编入口

```
.section .text.trampoline
.code64

.global syscall_entry

syscall_entry:
    swaps
    mov qword ptr gs:[offset __PERCPU_USER_RSP], rsp
    mov rsp, qword ptr gs:[offset __PERCPU_TSS +
{tss_rsp0_offset}]
    sub rsp, 8
    push qword ptr gs:[offset __PERCPU_USER_RSP]
    push r11
    sub rsp, 8
    push rcx
    sub rsp, 2 * 8
    push rax
    .....(此处省略)
    pop rax
    add rsp, 7 * 8
    mov rcx, [rsp - 5 * 8]
    mov r11, [rsp - 3 * 8]
    mov rsp, [rsp - 2 * 8]
    swaps
    sysretq
```

该汇编实现通过swaps切换到内核GS基址，然后把用户的RSP等必要上下文保存到percpu区或栈上，构造起供高层语言handler使用的TrapFrame。关键步骤包括按照ABI保存必要的通用寄存器、把用户提供的CR3与内核目标栈地址暂存到预定位置，再通过mov cr3, r13/mov rsp, r14等指令完成地址空间与栈的切换，随后调用内核的syscall handler。handler处理结束后，要根据返回的约定再恢复用户的CR3、寄存器与栈，并通过sysretq返回用户态。

4.5 ACPI/AML 与 PCI 的探测方式

x86_64 的设备发现以ACPI[1]为核心：先搜索RSDP并解析SDT（包括MADT/MCFG/SPCR/HPET等），从表中提取中断控制器、串口、HPET与PCIMCFG（MCFG指定了ECAM区域）。在MCFG存在时优先使用ECAM进行PCI总线扫描，否则采取默认ECAM基址或基于IO的枚举策略。

表 4-7 PCI ECAM 枚举与 MCFG 处理

```
pub fn enumerate_pci_devices<H: ::acpi::Handler>(tables:
&::acpi::AcpiTables<H>) -> Vec<CommonDeviceType> {
    let mut devices = Vec::new();
    if let Ok(pci_regions)
= ::acpi::platform::PciConfigRegions::new(tables) {
        for region in pci_regions.regions.iter() {
            let bus_count = (region.bus_number_end -
region.bus_number_start) as usize + 1;
            let base = region.base_address as usize;
            let size = bus_count << 20;
            let address_range =
PhysAddr::from(base)..PhysAddr::from(base + size);
            devices.push(ecam_device(CommonDeviceInfo
{ address_range: address_range.clone(), locator:
DeviceLocator::Mmio(address_range), irq: None, compatible:
Some("pci_ecam".into()), }));
        }
        return devices;
    }
    // 兜底 ECAM
    let base = platform::config::PCI_ECAM_BASE;
    let size = 0x1000_0000usize;
    let address_range =
PhysAddr::from(base)..PhysAddr::from(base + size);
```

```
        devices.push(ecam_device(CommonDeviceInfo { address_range:
address_range.clone(), locator:
DeviceLocator::Mmio(address_range), irq: None, compatible:
Some("pci_ecam".into()), }));
        devices
    }
}
```

MCFG表中列出的ECAM区域通常包含segment与bus range，内核应把每个区域视为一段连续的PCI配置空间并据此进行MMIO访问与设备发现。优先使用ACPI中的MCFG可以避免错误地假定固定基址，并能正确处理多段ECAM情况；当MCFG不存在时用平台默认值作为回退可以保持基本兼容性，但应在日志中标注并在驱动或固件期望不同时向上层报告。

第 5 章 vDSO 实现与用户态集成

本章介绍vDSO（virtual Dynamically linked Shared Object）在本项目中的实现思路与工程细节，重点讨论构建与产物、内核侧的映射与刷新机制、导出API与用户态解析、以及性能与一致性之间的权衡。

5.1 vdso_crate_template 构建链路与产物

本项目不是抽象地“用模板构建vDSO”，而是直接接入vdso_crate_template提供的build_vdso与vdso_helper两组能力：前者负责把vdso_impl编译为可映射的so与api库，后者负责在vDSO实现中生成vVAR访问宏与数据布局。这样内核侧与用户态侧共享同一份接口定义，避免手写胶水代码引入ABI偏差。

构建入口由vdso_builder执行。它读取ARCH或VDSO_ARCH，组装BuildConfig并调用build_vdso_tool，默认把源目录指向vdso/vdso_impl，输出目录指向build/vdso。与此同时，vdso_impl通过vdso_helper中的vvar_data!与get_vvar_data!声明共享快照区，并在api模块导出__vdso_clock_gettime等符号。

Listing 6展示了vdso_crate_template在本项目中的实际接入点（构建配置+vVAR布局），说明“构建链路”和“运行时共享数据布局”是如何落到代码上的：

表 5-1 vdso 构建配置

```
use build_vdso::{build_vdso as build_vdso_tool, BuildConfig};

fn build_vdso_for_arch(target_arch: TargetArch, repo_root:
&Path) {
    let mut config = BuildConfig::new("../vdso/vdso_impl",
"vdso_impl");
    config.arch = target_arch.build_arch().to_string();
    config.out_dir = "../build/vdso".to_string();
    config.so_name = "libvdso".to_string();
    config.api_lib_name = "vdso_api".to_string();
    build_vdso_tool(&config);
}
```

上面这组代码体现了本项目对vdso_crate_template的实际使用方式：构建阶段由build_vdso产出vDSO工件，运行时阶段由vdso_helper约束vVAR布局，二者共同保证了内核刷新与用户读取在字段偏移和ABI上的一致性。

Listing 7展示了vVAR布局与基于seqcount的无锁读取实现的关键片段，这一机制允许用户态在不进入内核的情况下获得一致的时间快照：

表 5-2 vdso_crate_template 接入与 vVAR 定义

```
vvar_data! {
    seq: usize,
    realtime_ns: usize,
    monotonic_ns: usize,
}

pub(crate) fn read_clock_timespec(clock_id: usize) -> Option<TimeSpec> {
    macro_rules! read_snapshot {
        ($field:ident) => {{
            loop {
                let seq = unsafe
                { core::ptr::read_volatile(get_vvar_data!(seq)) };
                if seq & 1 != 0 { core::hint::spin_loop(); continue; }
                let nanos = unsafe
                { core::ptr::read_volatile(get_vvar_data!($field)) };
                let seq_after = unsafe
                { core::ptr::read_volatile(get_vvar_data!(seq)) };
                if seq == seq_after { break Some(TimeSpec { tv_sec: nanos /
                1_000_000_000, tv_nsec: nanos % 1_000_000_000 }); }
            }
        }};
    }

    match clock_id {
        CLOCK_REALTIME => read_snapshot!(realtime_ns), CLOCK_MONOTONIC =>
        read_snapshot!(monotonic_ns)
        _ => None
    }
}
```

上面实现依赖奇偶版本号（seqcount）语义：读者在读数据前后检查版本号是否相等，从而判断读取期间是否发生了写入。如果版本号不一致则重试或返回失败，这种无锁的快路径在高频读场景（如clock_gettime）能显著降低延迟。

5.2 内核侧 MemIf 实现与 vDSO 装载

vDSO引入不仅是用户态代码和构建器的工作，还依赖内核侧MemIf实现来完成地址空间预留、物理页分配、页表映射、权限切换和用户虚地址到内核可访问地址的

转换。本项目在KernelVdsoMem中实现了vdso_api::MemIf，并在init_vdso中调用vdso_api::map_so完成装载。

表 5-3 内核侧 MemIf 实现与装载入口

```
pub fn init_vdso() {
    VDSO_LOADED.store(true, Ordering::SeqCst);
    let vdso_base = vdso_api::map_so(mem::kernel_page_table_token()) as
    usize;
    unsafe { vdso_api::init_vdso_vtable(vdso_base as u64) };
    VDSO_LOADED.store(false, Ordering::SeqCst);
}

struct KernelVdsoMem;

#[crate_interface::impl_interface]
impl MemIf for KernelVdsoMem {
    fn valloc(vspace: usize, size: usize) -> *mut u8 { ... }
    fn ppage_alloc(size: usize) -> vdso_api::PhysPagePtr { ... }
    fn map(vspace: usize, vaddr: *mut u8, ppage: vdso_api::PhysPagePtr,
    size: usize, flags: vdso_api::MappingFlags) { ... }
    fn change_protect(vspace: usize, vaddr: *mut u8, size: usize, flags:
    vdso_api::MappingFlags) { ... }
    fn get_kernel_vaddr(vspace: usize, vaddr: *mut u8) -> *mut u8 { ... }
    fn ppage_clone(ppage: vdso_api::PhysPagePtr) -> vdso_api::PhysPagePtr
    { ... }
}
```

这一实现把“平台无关的vDSO装载流程”与“平台相关的页表/物理页操作”解耦：vdso_api只依赖MemIf抽象，内核通过KernelVdsoMem把抽象绑定到现有内存管理与task域RPC能力，从而同时覆盖内核地址空间映射与用户地址空间映射两条路径。

5.3 内核侧映射与刷新策略

内核负责将vDSO的ELF段映射到进程的虚拟地址空间，并把vVAR区的虚拟地址通过进程上下文或全局记录暴露给内核写入路径。内核写入vVAR的基本约定是使用seqcount风格的写序列：先把版本号置为奇数表示“写入中”，写入数据字段并执行必要的内存屏障，最后把版本号置为偶数表示“写入完成”。这种写法保证了用户态的读者在检测到版本号为偶数且前后版本一致时读取到的就是完整一致的快照。

表 5-4 内核侧 vVAR 刷新

```

let data = unsafe { &mut *(vvar_base as *mut VvarData) };
let seq = data.seq.wrapping_add(1) | 1;
data.seq = seq;
core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::Release);
data.realtime_ns = realtime_ns;
data.monotonic_ns = monotonic_ns;
core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::Release);
data.seq = seq.wrapping_add(1);

```

vDSO的部署还需要内核在进程初始化时将vDSO ELF的入口地址通过auxv注入到用户态，供C运行时在启动阶段解析并优先绑定vDSO提供的符号；若解析失败，C运行时应回退到传统的syscall路径以保证功能完整性。

5.4 userlib vdso 加载

用户态的vDSO加载逻辑集中在userlib中，本项目的userlib通过解析程序初始栈上的auxv（AT_SYSINFO_EHDR）获得vDSO ELF的基址并初始化vdso_api的vtable，从而在后续调用中可以直接调用由vDSO导出的快速路径接口（例如__vdso_clock_gettime）。当vDSO不可用或调用失败时，userlib会回退到传统的syscall实现。

下面摘录了 user/userlib/src/vdso.rs 的关键函数，展示userlib如何解析auxv、初始化vDSO基址并优先使用vDSO快路径：

表 5-5 userlib vDSO 加载与调用

```

static VDSO_BASE: AtomicUsize = AtomicUsize::new(0);
static VDSO_READY: AtomicUsize = AtomicUsize::new(0);

pub fn init_from_stack(argc_ptr: usize) {
    if let Some(base) = parse_vdso_base(argc_ptr) {
        init(base);
    }
}

pub fn init(base: usize) {
    if !is_valid_vdso_base(base) { return; }
    VDSO_BASE.store(base, Ordering::Release);
    unsafe { vdso_api::init_vdso_vtable(base as u64) };
    VDSO_READY.store(1, Ordering::Release);
}

```

```
pub fn clock_gettime_vdso(clk: usize, ts: &mut TimeSpec) -> bool {
    if VDSO_READY.load(Ordering::Acquire) != 0 &&
VDSO_BASE.load(Ordering::Acquire) != 0 {
        let mut vdso_ts = vdso_api::TimeSpec::default();
        let ret = vdso_api::__vdso_clock_gettime(clk, &mut vdso_ts as *mut
_);
        if ret == 0 {
            ts.tv_sec = vdso_ts.tv_sec;
            ts.tv_nsec = vdso_ts.tv_nsec;
            return true;
        }
    }

    false
}

pub fn clock_gettime(clk: usize, ts: &mut TimeSpec) -> bool {
    if clock_gettime_vdso(clk, ts) { return true; }
    sys_clock_gettime(clk, ts as *mut TimeSpec as *mut u8) == 0
}
```

在进程启动时userlib从初始栈解析AT_SYSINFO_EHDR并缓存vDSO基址，然后用vdso_api::init_vdso_vtable()把vDSO导出的符号绑定到用户态可调用的函数指针或跳转表。当快路径调用失败时，回退到sys_clock_gettime系统调用，保证功能完整性。

第6章 验证与评估

本章基于一次完整运行的实验日志，系统性论述将系统迁移到x86_64平台后在功能性与性能方面的验证结果。引言部分概述研究目的与方法学框架；随后分别就测试设计、功能验证、性能度量、方法学评述与复现性建议展开论述，并在结论中总结主要发现与后续工作方向。

6.1 测试设计

在测量用户态读取时钟的快路径（vDSO）相对于传统系统调用路径（raw syscall）时，必须避免用被测时间源自身来度量其耗时，这会因分辨率和返回速度差异产生误导性零值或非物理性偏差。为此，本研究采用两阶段方法：先进行一致性验证，确认vDSO与raw在绝对时间上无不可接受差异；随后采用独立基准的连续采样窗口进行性能测量。连续采样窗口在窗口前后调用原始基准打点函数记录纳秒级时间，窗口内仅执行被测调用；窗口总耗时除以调用次数得出单次平均耗时，从而尽量降低被测源自身的量化误差对性能评估的影响。

在具体实现中，研究团队为降低系统噪声的影响采用了预热（warmup）步骤以去除冷启动与缓存效应的短期偏差，并在每组测量前执行固定次数的预热迭代。此外，测量实现中使用了黑盒提示（core::hint::black_box）以避免编译器在循环中进行不当优化，这一点在微基准测试中尤为重要。样本组选择（64、128、256、512、1024）旨在覆盖从小窗口到较大窗口的尺度，以观察窗口大小对测量稳定性与信噪比的影响。测量阈值（本实验中采用10ms作为一致性判断的上限）基于虚拟化环境下观测到的瞬时波动范围确定；在更稳定的硬件平台上，该阈值可以相应收紧。

功能测试方面，本研究采用端到端与单元测试相结合的策略，以保证系统从固件引导、内核初始化到用户态服务的各个子系统在功能语义上保持一致。端到端测试通过自动化的引导运行框架把系统引导到多核配置并运行完整的用户态测试套件，记录启动日志以验证frame allocator、内核地址空间构建、域注册与加载、以及用户态进程能否成功解析auxv并定位vDSO；单元与集成测试则覆盖vdso相关的边界条件与错误路径，包括验证vdso映射失败时userlib能正确回退到syscall、验证vdso_api的vVAR布局在内核与vDSO间保持二进制兼容、验证MemIf实现对于valloc/ppage_alloc/map/change_protect/get_kernel_vaddr/ppage_clone等接口在内核与

task域之间的正确行为。为捕获并回放较难触发的竞态与权限错误，测试框架还集成了针对中断与并发写入场景的模拟模块，用以在受控条件下触发vVAR并发更新、验证seqcount语义在读者/写者竞争下的健壮性。测试结果的判定主要基于日志中一致性的断言以及若干关键二进制断言（例如vDSO ELF校验、符号解析成功与否、用户态vdso_api初始化返回值），并辅以自动化复测与回归测试以保证在代码变更后功能不回退。

6.2 功能验证与运行证据

功能验证基于系统从引导到进入用户态测试套件并成功呈现交互shell的日志序列。以下摘录代表性日志片段，用以证明引导、内存与地址空间初始化、域注册与加载、以及用户态测试的顺利完成。

表 6-1 运行日志节选

```
[0] [x86_boot] main_entry magic=0x2badb002 mbi=0x9500
[0] Frame allocator init success
[0] Build kernel address space success
[0] <register domain>: scheduler, logger, fatfs, ramfs, devfs, procfs,
sysfs, pipefs, domainfs, vfs, task, syscall, uart16550, local_apic, io_apic,
cmos_rtc, virtio_net
[0] Load domains done
...
[syscall_all] start
[sys_poll] PASS
[sys_fs] PASS
[sys_link] PASS
[sys_proc] PASS
[sys_random] PASS
[sys_time] PASS
[sys_vdso_time_compare] PASS
[syscall_all] PASS
[Init] all tests passed
Alien:/#
```

上述片段按照时间序列表明了系统关键路径的成功执行，支持对功能正确性的断言。

值得指出的是，日志片段提供的是面向整链路的证据：启动阶段验证了引导加载器与内核早期初始化（如frame allocator、内核地址空间构建）是否正确完成；域注册与加载片段则验证了系统模块化加载逻辑与依赖解析；用户态测试的通过则体现了系统

调用转发、用户态库与运行时环境的协同正确性。尽管日志本身不能穷尽所有潜在的边界情况，但其连贯性与顺序性足以作为功能完整性的主要证据。

6.3 性能度量与分析

在一致性验证通过后，性能测量在多个样本组上分别对vDSO与raw syscall的平均单次耗时进行窗口测量。样本组的典型取值为64、128、256、512、1024。下列为运行日志中记录的原始测量行，作为后续统计分析的原始数据来源。

表 6-2 vdsO 与 syscall 性能对比表

样本数	重复次数	vDSO 平均(ns)	vDSO 标准差(ns)	raw 平均(ns)
64	5	1327	512	38860
128	5	511	64	35357
256	5	363	64	37914
512	5	282	16	36551
1024	5	286	32	45534

从表中可见，vDSO平均耗时处于数百至千纳秒量级，而raw平均耗时处于数万纳秒量级，因而vDSO相对于raw呈现显著加速；当在每组内部重复测量（repeats=5）并计算均值与标准差后，较大样本组（256、512、1024）显示出更稳定的raw均值但仍存在数千纳秒级的波动，而vDSO的标准差相对较小但并非恒定，说明尽管vDSO单次开销很小，其测量值对系统抖动和基准打点误差敏感。基于此结论，建议在后续工作中对每个样本组增加独立重复-run以获得统计置信区间，并在物理硬件上复验以排除虚拟化噪声对加速比估计的影响。

7. 下一步研究方向

7.1 协作调度完善（引入 vsched2）

在完成x86_64平台上的基础移植后，首要的拓展方向是将协作式调度（以vsched2为代表的机制）更深地融入到域生命周期与运行时治理中。具体工作包括设计并验证抢占与合作的混合策略、在域之间引入合理的优先级继承与迁移机制以减少优先级反转的风险、以及在多核和NUMA拓扑下评估调度器的可伸缩性和负载均衡能力。工程上需要将vsched2的调度语义与Alien的域隔离语义对齐，定义明确的调度契约和度量指标，并通过微基准与系统级负载测试验证在保持隔离性的前提下对高频系统调用和域间通信延迟的影响。最后，还应构建回退与监控手段，使得当协作调度在某些场景下不满足实时性或公平性要求时，系统能够安全地回退到更保守的调度策略。

7.2 引入用户态中断

将用户态中断作为下一阶段研究重点，可以在不牺牲隔离性的前提下进一步缩短某些I/O与事件处理路径的延迟，从而提升域间交互和高频事件响应的性能。该方向需要在内核与域之间设计安全的中断传递与授权机制，保证中断上下文与域运行状态之间的一致性，并处理好中断重入、优先级以及在驱动域或用户库中的可重入实现问题。工程实践上要解决中断来源验证、快速分发通道（可能借助vDSO/vVAR提供的元信息）以及在中断处理失败时的降级和恢复策略，同时评估用户态中断对调度器、抢占边界和现有回退路径（例如syscall回退）的交互影响。

7.3 在 x86_64 物理机上运行

将系统部署并运行在真实的x86_64物理硬件上，是验证工程可用性与鲁棒性的关键步骤。该工作包括硬件适配、镜像与引导介质准备、驱动与固件兼容性验证以及面向现场故障的调试与恢复流程设计。可以做的具体工作包括：构建针对x86_64的内核与用户镜像，准备可用于USB/SD或固态盘的引导映像，并在UEFI/BIOS环境中验证引导链（包括EFI可执行文件、引导参数与模块签名等）；按需启用或适配ACPI、APIC、PCI和IOMMU支持，以保证设备探测与中断路由的正确性；优先启用串口或网络控制台以便捕获早期启动日志，并在映像中集成可回退的启动项以便快速恢复；对关键外设（存储、网络、GPU、USB控制器等）做分阶段探测与驱动加载验证，记

录探测顺序、资源分配与代理注册的差异；为处理固件差异与厂商特性，建立可重放的“镜像+配置”脚本（例如使用`dd`/`mttools`写入引导介质、配置grub/EFI条目、设定kernel cmdline），并准备故障注入与重启恢复脚本以验证驱动域崩溃时的隔离与回退策略；在物理机上开展长期稳定性与性能测试（包括多核负载、I/O压力测试以及vDSO快路径在真实硬件上的延迟测量），将测试结果纳入回归套件以便持续验证；最后，把物理平台运行中收集的日志、固件信息与探测差异整理成可复用的硬件兼容性数据库，以指导后续设备支持和自动化测试。

结 论

本文以工程实现为主线，围绕将Alien系统从RISC-V平台迁移到x86_64并引入vDSO的设计展开研究和实现工作。在不破坏既有RISC-V路径的前提下，我们提出并实现了针对x86_64的平台抽象，完成了启动与早期内存管理、多核同步、中断与syscall适配等关键子系统的移植工作，构建了域加载与设备探测的工程链路，并实现了vDSO/vVAR的协作设计以提供用户态的低延迟快路径和可靠的回退机制。初步验证表明，所提出的抽象有助于减少架构分叉带来的维护负担，vDSO在常见的时钟与高频查询场景中带来了可观的延迟改善，而域隔离与驱动加载设计在多核环境下保持了基本的可用性与可恢复性。尽管当前实现仍以最小可运行目标为导向，存在设备支持不全、某些子系统需进一步扩展及更严格形式化验证不足等局限，但本研究为在多架构环境下实现轻量级隔离内核提供了切实的工程路径与实践经验，并为后续在协作调度、用户态中断、vDSO深度优化与更广泛设备支持等方向的研究奠定了基础。

参考文献

- [1] Rust OSDev Community. acpi: A library for parsing ACPI tables and AML[CP/OL]. [2026-05-18]. <https://github.com/rust-osdev/acpi>.
- [2] x2apic Developers. x2apic: A Rust interface to the x2apic interrupt architecture (version 0.4.1)[CP/OL]. [2026-05-18]. <https://docs.rs/x2apic/0.4.1/>.
- [3] x86_64 Contributors. x86_64: Support for x86_64 specific instructions, registers, and structures[CP/OL]. [2026-05-18]. https://docs.rs/x86_64/.
- [4] ArceOS Community. ArceOS: A modular operating system framework in Rust[CP/OL]. [2026-05-18]. <https://github.com/arceos/arceos>.
- [5] rosy233333. vdso_crate_template[CP/OL]. [2026-05-18]. https://github.com/rosy233333/vdso_crate_template.
- [6] Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer ‘s Manual Combined Volumes 3A, 3B, 3C, and 3D: System Programming Guide[R/OL]. (2026-03-31)[2026-05-19]. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [7] UEFI Forum. Advanced Configuration and Power Interface (ACPI) Specification, Version 6.5[S/OL]. (2022-08-29)[2026-05-19]. <https://uefi.org/specifications>.
- [8] PCI-SIG. PCI Firmware Specification Revision 3.2[S/OL]. (2015-01-26)[2026-05-19]. <https://pcisig.com/specifications>.
- [9] GNU GRUB Project. Multiboot Specification (version 0.6.96)[S/OL]. [2026-05-19]. <https://www.gnu.org/software/grub/manual/multiboot/multiboot.html>.
- [10] RISC-V International. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20211203[S/OL]. (2021-12-03)[2026-05-19]. <https://riscv.org/technical/specifications/>.
- [11] Linux Kernel Community. vdso(7) — Linux manual page[EB/OL]. (2026-03-20)[2026-05-19]. <https://manpages.debian.org/testing/manpages/vdso.7.en.html>.
- [12] Klein G, Elphinstone K, Heiser G, et al. seL4: formal verification of an OS kernel[C]//Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP 2009). Big Sky, Montana, USA: ACM, 2009: 207-220.
- [13] Liedtke J. On μ -kernel construction[C]//Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP 1995). Copper Mountain Resort, CO: ACM, 1995: 237-250.
- [14] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: securing the foundations of the Rust programming language[J]. Proceedings of the ACM on Programming Languages (POPL), 2018, 2: 1-34.
- [15] Klabnik S, Nichols C. The Rust Programming Language[M]. 2nd ed. San Francisco: No Starch Press, 2023.
- [16] The Rust Project. The Rust Reference[EB/OL]. [2026-05-19]. <https://doc.rust-lang.org/reference/>.